

Class-Based Testbench

AXI-Lite Slave IP with Memory

ECE-593: Fundamentals of Pre-Si Validation

Portland State University

Team: Patrick, Pratibha

1. Overview

This document describes a SystemVerilog class-based verification environment for the AXI4-Lite Slave IP. The testbench follows a layered architecture inspired by UVM principles, using mailboxes for inter component communication, virtual interfaces for DUT connectivity, and embedded functional coverage to drive closure.

2. Architecture

The environment is organized into seven source files, each encapsulating a single responsibility:

File	Class / Module	Responsibility
<code>axil_if.sv</code>	<code>axil_if</code>	AXI4-Lite interface: bundles all signals, defines master/monitor clocking blocks and a reset task.
<code>transaction.sv</code>	<code>transaction</code>	Data object holding all fields for a single read, write, or concurrent read+write (TXN_BOTH) operation, with SV randomization constraints.
<code>generator.sv</code>	<code>generator</code>	Produces directed and random transaction streams into a gen2drv mailbox.
<code>driver.sv</code>	<code>driver</code>	Bus Functional Model: pulls transactions from gen2drv and drives all five AXI4-Lite channels using the master clocking block.
<code>monitor.sv</code>	<code>monitor</code>	Passively observes the bus, reconstructs complete write and read transactions, and forwards them to the scoreboard. Hosts cycle-level functional coverage.
<code>scoreboard.sv</code>	<code>scoreboard</code>	Maintains a shadow memory model, checks every observed transaction, and hosts transaction-level functional coverage.

File	Class / Module	Responsibility
env.sv	env	Top-level container: instantiates all components, owns all mailboxes, and provides run / drain / stop / report lifecycle methods.
tests.sv	test_base + derived	Test library: base class with a standardized lifecycle and four concrete tests.
testbench_top.sv	testbench_top	Static top module: clock generation, interface and DUT instantiation, and sequential test execution.

3. Component Detail

3.1 Interface (axil_if)

The interface is parameterized on DATA_WIDTH and ADDR_WIDTH and accepts the clock as an input port. It defines two clocking blocks:

- Master_cb: drives stimulus to the DUT (outputs: awvalid, awaddr, awprot, wvalid, wdata, wstrb, bready, arvalid, araddr, arprot, rready, resetn).
- Monitor_cb: samples all signals with #1step input skew for glitch-free observation.

A built-in do_reset() task asserts reset for 10 cycles, clears all master outputs, then deasserts for 5 cycles. Two modports (MASTER, MONITOR) expose the appropriate clocking block to each component.

3.2 Transaction (transaction)

The transaction class is the data token passed between all components. Its key fields are:

- mode: enumeration: TXN_WRITE, TXN_READ, or TXN_BOTH.
- addr, wdata, wstrb, prot: randomized stimulus fields with constraints enforcing 32-bit word alignment and a non-zero strobe on writes.
- rdata, bresp, rresp: response fields populated by the monitor after observing the DUT.

The c_mode_random constraint limits random generation to TXN_WRITE and TXN_READ; TXN_BOTH is injected only via the generator's directed API.

3.3 Generator

The generator sends directed transactions first (from the directed_txn_q queue), then the configured number of random transactions. Convenience methods add_write(), add_read(), and add_both() let tests build the directed queue without constructing transaction objects manually. Address randomization is constrained to the valid memory window (word-aligned, within MEM_DEPTH words).

3.4 Driver

The driver operates as an AXI4-Lite master BFM. For write transactions it forks two parallel threads to handle the AW and W channels independently (matching real AXI master behaviour), then sequentially handles the B response. For read transactions it drives AR, waits for the handshake, then asserts rready. For TXN_BOTH it aligns AW, W, and AR to the same clock edge using the maximum of their individual random delays, waits for all three ready signals simultaneously, then handles B before R responses.

Five configurable delay knobs (`max_aw_delay`, `max_w_delay`, `max_ar_delay`, `max_b_delay`, `max_r_delay`) allow back-pressure stress testing without requiring a separate test class.

3.5 Monitor

The monitor runs five concurrent threads inside a `fork/join_none` block:

- `monitor_aw()`: captures AW-channel handshakes into an internal `aw_mbx`.
- `monitor_w()`: captures W-channel handshakes into an internal `w_mbx`.
- `merge_write()`: pairs AW and W objects, waits for the B handshake to obtain `bresp`, then sends a merged transaction to the scoreboard.
- `monitor_read()`: captures the AR handshake then the R handshake and sends the complete read transaction to the scoreboard.
- `cycle_coverage()`: a continuous cycle-level tracker (described in Section 4).

3.6 Scoreboard

The scoreboard maintains a shadow memory array (`shadow_mem`) that mirrors the DUT's memory. On every observed write it applies the byte-strobe mask to update the shadow at the word address computed from `addr`. On every observed read it compares `rdata` against the shadow value and flags any mismatch with `$error`. `BRESP` and `RRESP` are independently checked for OKAY (2'b00); any other code is also flagged. Statistics (`writes_checked`, `reads_checked`, `errors`) are reported at the end of simulation.

3.7 Environment

The `env` class owns all three mailboxes and instantiates the generator, driver, monitor, and scoreboard in its `init()` method. Its `run()` method forks all four component `run()` tasks with `join_none`. The `drain()` method polls the scoreboard's transaction counters and blocks until all expected transactions have been checked (with a configurable cycle timeout to prevent infinite hangs). `stop()` calls `disable fork` to cleanly terminate all threads.

3.8 Test Library

All tests extend `test_base`, which enforces the lifecycle: `configure()` -> `directed_phase()` -> `do_reset()` -> `env.run()` -> `env.drain()` -> `env.stop()` -> `env.report()`. The four concrete tests are:

Test	Purpose
<code>test_random</code>	20 random TXN_WRITE / TXN_READ transactions.
<code>test_wr_rd_same</code>	Directed: writes a known pattern (0xDEAD_000N) to 8 word-aligned addresses then reads each back, verifying basic write-then-read coherence.
<code>test_byte_strobe</code>	Directed: writes 0xAABBCCDD with full strobe, then overwrites each byte lane individually, then reads back expecting 0x12345678 — verifying the DUT's byte-lane masking.
<code>test_concurrent_rw</code>	Directed: issues 9 TXN_BOTH transactions, exercising simultaneous AW/W/AR channel arbitration and B/R response ordering.

4. Functional Coverage

Coverage is split between two components based on signal visibility. The monitor holds cycle-level covergroups; the scoreboard holds transaction-level covergroups.

ID	Covergroup	Owner	What is Measured
FV-001	<code>cg_reset</code>	Monitor	Reset asserted and de-asserted states.
FV-002/003	<code>cg_aw_w_hs</code>	Monitor	AW-first, W-first, and concurrent AW+W handshake orderings.
FV-005	<code>cg_bresp</code>	Scoreboard	BRESP codes: OKAY and SLVERR.
FV-006	<code>cg_wstrb</code>	Scoreboard	All 15 valid WSTRB byte-lane patterns (single, two, three, and full byte writes).
FV-007/010	<code>cg_oor</code>	Scoreboard	Cross of in-range/out-of-range address vs. write/read operation type and response code.
FV-008	<code>cg_ar_hs</code>	Monitor	AR-to-R channel latency binned: 1, 2, 3, 4-7, 8-15, and 16+ cycles.
FV-009	<code>cg_rresp</code>	Scoreboard	RRESP codes: OKAY and SLVERR.
FV-011	<code>cg_fwd</code>	Monitor	Same-cycle AW and AR handshakes to the same word address (write-before-read forwarding scenario).
FV-012	<code>cg_addr</code>	Scoreboard	Word address distribution across four equal quarters of the memory map, plus out-of-range.

ID	Covergroup	Owner	What is Measured
FV-013	cg_bp_b / cg_bp_r	Monitor	BREADY and RREADY back-pressure stall duration binned: 0, 1-3, 4-7, 8-15, and 16+ cycles.
FV-014	cg_b2b	Scoreboard	Back-to-back transaction ordering: write-write, write-read, read-write, and read-read pairs.